OrchIso

Orchestral Isolation Tool

Two-Stage Audio Source Separation utilizing Demucs and Spectrogram U-Net

by: tembok ratapan solo

Final Project KCVanguard

2026

1. Overview

OrchIso adalah tool berbasis machine learning untuk mengisolasi bagian orkestra dari sebuah lagu audio. Tool ini menggunakan pipeline dua tahap: pertama memisahkan stem menggunakan Demucs, kemudian memperbaiki hasil separasi menggunakan model Spectrogram U-Net yang dapat dilatih sendiri.

Tujuan utama OrchIso adalah mengekstrak stem orkestra (strings, brass, woodwind, dan instrumen klasik lainnya) dari lagu yang sudah dicampur, terutama lagu pop atau klasik yang mengandung elemen orkestra.

1.1 Fitur Utama

- Pipeline dua tahap: Demucs (Stage 1) + Spectrogram U-Net (Stage 2)
- Model berbasis spectrogram domain — menghindari distorsi waveform
- Mask-based output: model tidak bisa menghasilkan noise yang tidak ada di input
- Support dataset MP3, WAV, FLAC
- Training dengan early stopping otomatis dan learning rate scheduler
- Overlap-add processing dengan Hann window untuk transisi chunk yang mulus
- Integrasi Google Drive untuk penyimpanan model dan dataset

1.2 Arsitektur Pipeline

Pipeline OrchIso terdiri dari dua tahap utama:

Tahap	Komponen	Input	Output
Stage 1	Demucs (mdx_extra)	File lagu MP3/WAV	Stem: vocals, bass, drums, other
Stage 2	Spectrogram U-Net 2D	Stem 'other' dari Demucs	Orkestra yang sudah dibersihkan

---

2. Requirements

2.1 Platform

- Google Colab (direkomendasikan) dengan GPU T4 atau lebih tinggi
- Google Drive untuk penyimpanan dataset dan model
- Browser modern (Chrome, Firefox, Edge)

2.2 Library Python

Library	Versi	Fungsi
PyTorch	>=2.0	Framework deep learning utama
torchaudio	>=2.0	Loading audio, STFT, Griffin-Lim
Demucs	>=4.0	Stage 1: source separation
ffmpeg	>=4.4	Audio encoding/decoding
numpy	>=1.21	Operasi numerik

2.3 Dataset

- Minimal 10 pasang file audio (dirty/clean) untuk sanity check
- Direkomendasikan 30-50+ pasang untuk hasil yang layak dipakai
- Format yang didukung: MP3, WAV, FLAC
- Durasi ideal per file: 3-8 menit

---

3. Persiapan Dataset

3.1 Struktur Folder

Buat struktur folder berikut di Google Drive:

```
MyDrive/
└── orchiso_dataset/
    ├── dirty/     ← stem 'other' dari Demucs (input model)
    └── clean/     ← orkestra murni (target model)
```

Penting: nama file di folder dirty/ dan clean/ harus sama persis, termasuk huruf besar/kecil dan ekstensi.

Contoh pasangan yang valid:

```
dirty/beethoven_op18.mp3  ↔  clean/beethoven_op18.mp3
dirty/mozart_k331.wav    ↔  clean/mozart_k331.wav
```

3.2 Apa itu Dirty dan Clean?

Tipe	Deskripsi	Cara Mendapatkan
dirty	Stem 'other' hasil Demucs dari lagu full — mungkin masih mengandung bleed dari instrumen lain	Jalankan Demucs pada lagu full, ambil file other.wav
clean	Orkestra murni tanpa bleed dari instrumen lain	Rekaman orkestra terpisah, atau stem resmi dari album

3.3 Sumber Dataset yang Direkomendasikan

- MUSDB18 — 150 lagu dengan stem terpisah (vocals, bass, drums, other)
- Looperman — stem royalty-free yang bisa diunduh gratis
- Rekaman orkestra klasik dengan stem resmi (beberapa album Final Fantasy, film scores)
- Dataset MIDI klasik yang dirender dengan soundfont berkualitas tinggi

---

4. Setup dan Instalasi

4.1 Persiapan Google Colab

1. Buka Google Colab di colab.research.google.com
2. Upload file notebook OrchIso.ipynb
3. Aktifkan GPU: Runtime → Change runtime type → T4 GPU
4. Jalankan cell secara berurutan dari atas ke bawah

4.2 Instalasi Dependency (Cell 1)

Cell pertama menginstall semua library yang dibutuhkan:

```
!pip install -q demucs
!pip install -q torch torchaudio
!apt-get install -y -q ffmpeg
```

Proses ini membutuhkan waktu sekitar 2-3 menit. Tunggu hingga muncul konfirmasi sebelum menjalankan cell berikutnya.

4.3 Mount Google Drive (Cell 2)

Cell kedua menghubungkan Colab dengan Google Drive dan memverifikasi dataset:

```
drive.mount('/content/drive')
```

Saat pertama kali dijalankan, akan muncul popup untuk login Google. Setelah berhasil, cell akan menampilkan jumlah pasangan file yang ditemukan.

---

5. Arsitektur Model

5.1 Spectrogram U-Net 2D

OrchIso menggunakan arsitektur U-Net 2D yang beroperasi di spectrogram domain, bukan waveform domain. Pendekatan ini dipilih karena:

- Spectrogram merepresentasikan audio sebagai gambar 2D (frekuensi × waktu), lebih mudah dipelajari model
- Pola orkestra (strings = garis horizontal stabil di frekuensi menengah) sangat jelas secara visual di spectrogram
- Model berbasis waveform 1D rentan menghasilkan high-frequency noise (zero-crossing tinggi)

5.2 Mask-Based Approach

Alih-alih langsung menghasilkan waveform bersih, model memprediksi mask (nilai 0-1) yang dikalikan dengan spectrogram input:

```
output = mask × input_spectrogram
```

Pendekatan ini secara matematis tidak mungkin menghasilkan frekuensi yang tidak ada di input — mengeliminasi sumber distorsi utama.

5.3 Struktur Jaringan

Komponen	Layer	Channels
Encoder 1	ConvBlock2D	2 → 16
Encoder 2	ConvBlock2D + MaxPool2D	16 → 32
Encoder 3	ConvBlock2D + MaxPool2D	32 → 64
Encoder 4	ConvBlock2D + MaxPool2D	64 → 128
Bottleneck	ConvBlock2D + MaxPool2D	128 → 256
Decoder 4	ConvTranspose2D + ConvBlock2D	256 → 128
Decoder 3	ConvTranspose2D + ConvBlock2D	128 → 64
Decoder 2	ConvTranspose2D + ConvBlock2D	64 → 32
Decoder 1	ConvTranspose2D + ConvBlock2D	32 → 16
Output	Conv2D + Sigmoid	16 → 2 (mask)

Total parameter model: 1,943,922

5.4 STFT Parameter

Parameter	Nilai	Fungsi
N_FFT	2048	Resolusi frekuensi — jumlah titik FFT
HOP_LENGTH	512	Step antar frame STFT
WIN_LENGTH	2048	Panjang window Hann
SAMPLE_RATE	44100 Hz	Sample rate audio target
CHUNK_SIZE	176400 sampel	4 detik audio per chunk (44100 × 4)

---

6. Training

6.1 Hyperparameter

Parameter	Nilai	Keterangan
CHUNKS_PER_FILE	30	Chunk acak yang diambil per pasang file
BATCH_SIZE	6	Jumlah sampel per iterasi training
EPOCHS	30	Maksimum epoch training
LR	1e-3	Learning rate awal (Adam optimizer)
EARLY_STOP	7	Stop jika val loss tidak membaik selama 7 epoch
Train/Val split	80% / 20%	Pembagian dataset untuk training dan validasi

6.2 Loss Function

OrchIso menggunakan L1 Loss (Mean Absolute Error) di spectrogram domain:

```
loss = mean(|predicted_spectrogram - target_spectrogram|)
```

L1 Loss dipilih karena lebih robust terhadap outlier dibanding L2 Loss, menghasilkan audio yang lebih natural.

6.3 Learning Rate Scheduler

ReduceLROnPlateau digunakan untuk menyesuaikan learning rate secara otomatis:

- Jika val loss tidak turun selama 3 epoch berturut-turut, LR dikurangi 50%
- LR awal: 1e-3 → setelah trigger pertama: 5e-4 → setelah trigger kedua: 2.5e-4

6.4 Hasil Training (30 File Dataset)

Berikut adalah log training yang dicapai dengan 30 pasang file dataset:

Epoch	Train Loss	Val Loss	Keterangan
1	0.10536	0.08568	Best
2	0.09589	0.08350	Best
5	0.08342	0.07669	Best
6	0.08336	0.06922	Best
10	0.07505	0.09405	LR → 5.0e-04
13	0.07237	0.06028	Best
17	0.06904	0.07719	LR → 2.5e-04
20	0.06857	0.06937	Early Stop

Best val loss yang dicapai: 0.06028 pada epoch 13. Model di epoch ini yang disimpan dan digunakan untuk inferensi.

6.5 Interpretasi Val Loss

Val Loss	Interpretasi
> 0.10	Model belum belajar, masih awal training
0.05 – 0.10	Mulai ada pola, hasil audio masih kasar
0.01 – 0.05	Bagus — hasil audio mulai bersih
< 0.01	Sangat bagus, waspadai overfitting

---

7. Cara Menggunakan OrchIso

7.1 Pertama Kali (Training Baru)

5. Jalankan Cell 1: install dependency
6. Jalankan Cell 2: mount Drive, pastikan dataset terdeteksi
7. Jalankan Cell 3: definisi model dan hyperparameter
8. Jalankan Cell 4: definisi dataset loader
9. Jalankan Cell 5: mulai training (estimasi 20-40 menit dengan T4 GPU)
10. Jalankan Cell 6: upload lagu yang ingin diproses, tunggu Demucs selesai
11. Jalankan Cell 7: proses dengan model U-Net
12. Jalankan Cell 8: download hasil

7.2 Sesi Berikutnya (Model Sudah Terlatih)

Jika model sudah pernah ditraining dan tersimpan di Drive, cukup jalankan:

13. Cell 1: install dependency (wajib tiap sesi baru)
14. Cell 2: mount Drive
15. Cell 3: definisi model (wajib tiap sesi baru)
16. Cell 6: upload lagu, skip Cell 4 dan 5
17. Cell 7: load model dari Drive, proses lagu
18. Cell 8: download hasil

Di Cell 7, pastikan ada baris berikut untuk load model dari Drive:

```
model.load_state_dict(torch.load(
    f'{DRIVE_DATASET}/orchestra_spectrogram_best.pth'
))
```

7.3 Menambah Dataset dan Training Ulang

Jika ingin meningkatkan kualitas dengan menambah data:

19. Tambahkan file baru ke folder dirty/ dan clean/ di Drive
20. Jalankan semua cell dari awal (Cell 1 hingga Cell 5)
21. Model lama akan ditimpa dengan model yang baru dan lebih baik

---

8. Pilihan Model Demucs

OrchIso menggunakan Demucs sebagai Stage 1 untuk separasi stem awal. Tersedia beberapa pilihan model:

Model	Stems	Kecepatan	Rekomendasi Penggunaan
htdemucs	4 (vocals, bass, drums, other)	Cepat	Default, titik awal yang baik
htdemucs_ft	4 (vocals, bass, drums, other)	Sedang	Kualitas sedikit lebih baik
mdx_extra	4 (vocals, bass, drums, other)	Sedang	Direkomendasikan untuk stem 'other'
htdemucs_6s	6 (+ guitar, piano)	Lambat	Jika piano/gitar ingin dipisah otomatis

OrchIso saat ini menggunakan mdx_extra sebagai default karena menghasilkan stem 'other' yang lebih bersih untuk konten orkestra.

---

9. Troubleshooting

9.1 Error Umum

Error	Penyebab	Solusi
RuntimeError: window overlap add min	Bug PyTorch dengan center=False pada iSTFT	Gunakan Griffin-Lim reconstruction (sudah diimplementasikan di versi final)
NameError: name 'model' is not defined	Sesi Colab baru, variabel hilang	Jalankan ulang Cell 1, 2, dan 3 terlebih dahulu
FileNotFoundError: output/*.wav	process_audio crash diam-diam	Gunakan try/except untuk melihat error asli
RuntimeError: ReduceLROnPlateau verbose	PyTorch versi terbaru tidak support verbose=True	Hapus parameter verbose=True dari scheduler
IndentationError pada class	Indentasi kode tidak konsisten	Pastikan semua baris di dalam class menggunakan 4 spasi

9.2 Hasil Audio Masih Noise

Jika output masih mengandung noise atau distorsi berlebihan:

- Periksa jumlah dataset — minimal 30 pasang untuk hasil yang layak
- Cek sinkronisasi dirty/clean — selisih durasi tidak boleh lebih dari 1 detik
- Perhatikan val loss saat training — jika stagnan di atas 0.08, tambah data
- Coba model Demucs yang berbeda (htdemucs vs mdx_extra)
- Pastikan kualitas pasangan data sudah benar — clean harus benar-benar bersih

9.3 Training Terlalu Cepat Overfitting

Jika val loss naik setelah 2-3 epoch:

- Kurangi CHUNKS_PER_FILE dan EPOCHS
- Kurangi BATCH_SIZE
- Tambah EARLY_STOP agar model lebih sabar
- Solusi terbaik: tambah data training

Daftar Pusaka

Defossez, A., et al. (2021). Music Source Separation in the Waveform Domain. arXiv:1911.13254. https://arxiv.org/abs/1911.13254

Ronneberger, O., Fischer, P., & Brox, T. (2015). U-Net: Convolutional Networks for Biomedical Image Segmentation. MICCAI 2015. https://arxiv.org/abs/1505.04597

Stoter, F. R., et al. (2018). The 2018 Signal Separation Evaluation Campaign. LVA/ICA 2018. https://arxiv.org/abs/1804.10404

Griffin, D., & Lim, J. (1984). Signal estimation from modified short-time Fourier transform. IEEE Transactions on Acoustics, Speech, and Signal Processing. https://doi.org/10.1109/TASSP.1984.1164317

Luo, Y., & Mesgarani, N. (2019). Conv-TasNet: Surpassing Ideal Time-Frequency Magnitude Masking for Speech Separation. IEEE/ACM Transactions on Audio. https://arxiv.org/abs/1809.07454